ロボットによる6面外観検査ライン
持ち替えで「つかんだ面」まで見る

Physical AI 応用編1 最終課題　説明資料　／　MuJoCo シミュレーション＋画像認識AI（ResNet-18）

一言でいうと：コンベアを流れる切削部品を、カメラ3台とロボットアーム1台で6面すべて検査し、 不良品だけをNG箱へ、良品は**動いているレーンの元の位置へ戻す**ラインをシミュレーション上に作りました。 上面だけの検査では不良品の検出率が **20%**、持ち上げて底面と側面2面を見ても **77%** でした。 持ち替えで指に隠れた2面まで見ると **100%**（過検出 0%）になりました。 追加実験として、良品の画像だけで作れる異常検知を重ねると、学習していない種類の不良（角欠け）の検出が 20% から 80% に上がりました。 代わりに良品の誤NGが 16% 出ることも分かりました。 処理能力はアームを並べて上げる設計で、2台でレーンの搬送能力（6.4 個/分）にほぼ達すると試算しました。 撮影するノードと判定AIのノードは ROS 2 で分けてあり、判定を別のPCで動かせます。

1. 課題設定：どんな現場の、何を自動化するか

想定する現場	精密切削工場の出荷前工程。切削・洗浄を終えた部品を梱包前に全数検査する。
対象部品	アルミ（A6061）削り出しのセンサーハウジング。40 mm 角、上面に取付ねじ穴 1 か所。
見つけたい不良	打痕・傷（搬送や治具との接触）、油ジミ（洗浄残り）、角欠け。**6面のどこにでも出る。**
現状の課題	人が1個ずつ手に取り、回しながら6面を目視している。単調で疲れやすく、裏面・側面の見落としが起きやすい。
ねらい	人が手で回して見ている動きをロボットに置き換え、全6面を毎回同じ条件で撮って AI で判定する。

上から撮るだけの検査は簡単に自動化できますが、底面や側面の不良は写りません。 一方、ロボットが部品をつかむと、**指が触れている2面は必ず隠れます。**この「つかむと見えない面ができる」問題をどう解くかが、本課題の中心です。

2. ラインの設計

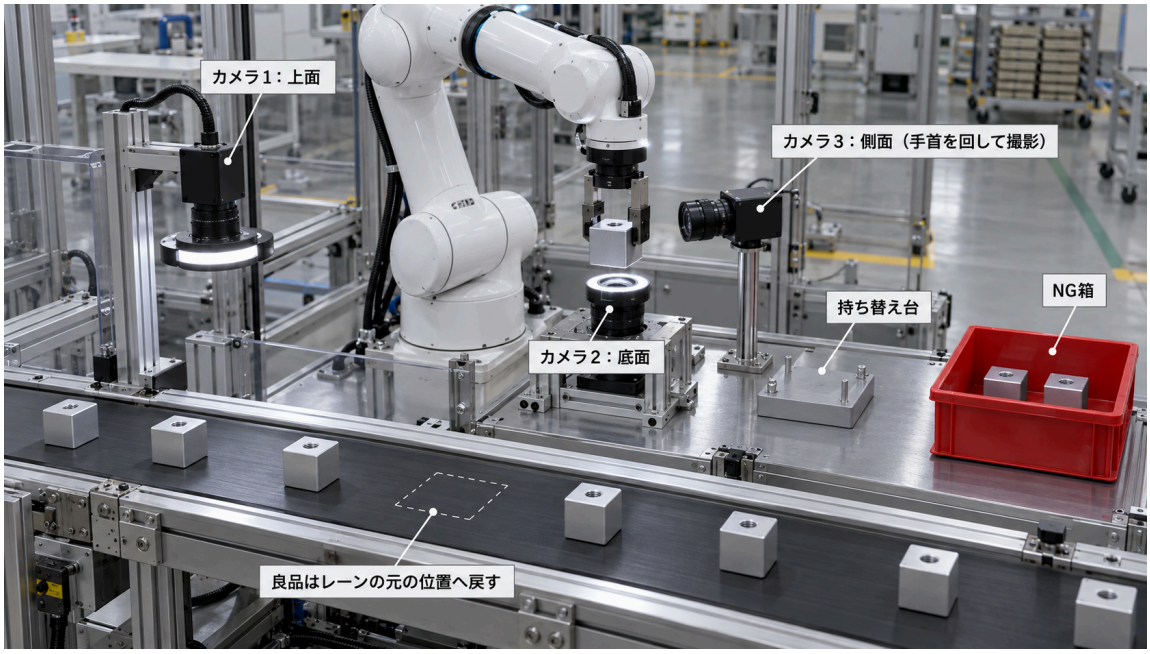


図1　ラインの構想図（生成AIで作成したイメージ）。実装はこの配置を MuJoCo 上に再現した。

1個あたりの流れ

1. 部品がレーンを流れてくる。**カメラ1（上面）**が通過中に上面を撮る。上面で不良ならそのまま NG 箱へ。
2. レーンが停止位置で止まり、アームが部品の左右（±x面）をつかんで持ち上げる。**レーンはすぐ再始動**し、後続の部品は流れ続ける。
3. 検査台で**カメラ2（下カメラ＋リング照明）**が底面を撮り、**カメラ3（横カメラ1台）**が手前の側面を撮る。手首（第7関節）だけを180°回し、反対側の側面も同じカメラ3で撮る。
4. 持ち替え台に置き、**90°回してつかみ直す**。最初に指で隠れていた2面を、カメラ3と手首の180°回転で撮る。
5. 6面すべて良なら、**動いているレーン上の元の位置**へ追従して戻す。どこかが NG なら NG 箱へ。不良が見つかった時点で残りの撮影は打ち切る。

設計判断：カメラは3台、能力はアームを並べて上げる

側面4面を4台のカメラで撮る方式も考えられます。しかし指で隠れた面は、カメラを増やしても撮れません。 持ち替えは必ず要るので、側面は1台のカメラに手首の回転で向けることにしました。処理能力が足りなければ、同じセル（アーム＋検査台）を並べて部品を分担させます。

方式	公開されている例	今回との関係
ロボットで向きを変える（1アーム＋少数カメラ）	スカイロジック「検査員ロボット」：アームが金属製品を横の固定カメラへ向け、上面・下面カメラを加えて6面化。ヒューブレイン「多面外観検査装置」：ハンドで回転・反転し 6面＋エッジ（□5〜50 mm）。	今回と同じ系統。40 mm 程度の多品種部品に向く。
専用搬送＋多数カメラ	オービット「ガラミちゃん」6面タイプ：ガラス円盤上で上下2台＋側面4台。チップ部品用の6面機（最大5,000個/分）。	小さな部品の大量生産向けの専用機。持ち替えは不要だが品種の変更に弱い。
ロボット2台で分担	HACARUS × FANUC CRX：搬送・反転用と検査用の2台構成。県の研究ではロボット2台化で 12秒→8秒（1.5倍）。	共有設備が律速になると台数に比例しない。今回の試算もレーン能力で頭打ちを示した。

出典：各社公式ページ・発表（スカイロジック、ヒューブレイン、HACARUS 2023-06-23、オービット 2026-10-23、新潟県工業技術総合研究所 2024-08）。詳細とURLはリポジトリの `research/report.md` 。

3. 実装



図2　デモ映像の画面。左：ライン全体、右上：3台のカメラ画像とAI判定、右中：検査中の部品の6面の状態（緑＝良、赤＝不良、灰＝未撮影）。

要素	内容
シミュレータ	MuJoCo 3.17。シーンは Python の MjSpec で組み立て（コンベア、カメラ3台と照明、検査台、持ち替え台、NG 箱、部品8個）。
ロボット	Franka Emika Panda（7軸＋2指グリッパ、MuJoCo Menagerie のモデル）。逆運動学は減衰最小二乗法＋冗長自由度で基準姿勢へ寄せる方式を自作。
コンベア	部品を一定速度（3 cm/s、ピッチ 28 cm）で運び、つかむ部品が停止位置に来たら止まる。上面カメラの通過時に撮影イベントを発生させる。
不良	6面のどこかに打痕・傷／油ジミ／角欠けを、位置・向き・大きさをランダムにして配置。
AI（判定）	面画像1枚を「良・打痕傷・油ジミ・角欠け」に分類する ResNet-18（ImageNet 学習済みから微調整）。1面でも不良確率が0.5を超えたら NG。 追加実験：良品の画像だけから作る異常検知（学習済み ResNet-18 の局所特徴を良品分だけ記憶し、いちばん見慣れない部分の距離で判定）。
ROS 2	ROS 2 Humble（macOS では RoboStack で導入）。ライン（シミュレーション＋ロボット）ノードが画像をトピックで送り、判定AIノードが結果を返す。関節の状態・作業の状態・部品ごとの結果も配信する。

工夫した点

1. **持ち替えで「つかんだ面」を見る。**検査台の持ち替え台に置き、手先を90°回してつかみ直すことで、最初に指で隠れた ±x 面を露出させる。撮影ログで、どの部品でも6面すべてが撮れていることを確認した。
2. **手首だけを180°回す。**腕全体で部品を裏返すと腕が大きく振れ、部品を落とした。第7関節だけを回すようにし、腕の姿勢を変えずに反対側の面を同じカメラへ向けた。

3. **関節の可動限界を見て、つかむ向きを選ぶ。**同じ面をつかむ向きは2通り（180°違い）ある。逆運動学で両方を試し、第7関節が限界から離れていて、さらに180°回す余裕が残る向きを選ぶ。横カメラも、第7関節が可動範囲の中央付近になる向き（検査台の −x 側）に置いた。 これで「限界に当たって部品を置ききれず落とす」失敗がなくなった。
4. **動いているレーンへ戻す（コンベア追従）。**つかんだ時点からのベルトの移動量を記録し、元の位置が今どこにあるかを毎周期計算して、その上へ降ろす。良品は元の並び順のまま次工程へ流れる。
5. **不良が見つかったら撮影を打ち切る。**NG が確定した時点で残りの面の撮影と持ち替えを省き、NG 箱へ直行する。
6. **ラインと同じ姿勢で学習画像を作る。**各カメラ・ロボット姿勢・手首の回転・持ち替え前後の向きを再現し、照明の明るさ、部品の色味、位置と角度のずれをランダムに変えて 16,000 枚を合成した。
7. **未知の不良への備えを試した。**現場では不良品の画像がほとんど集まらず、想定外の不良も出る。良品の画像だけで作れる異常検知を分類器に重ね、得られるもの（未知の不良の検出）と失うもの（良品の誤NG）を測った。異常検知の記憶には、ラインに良品を流して撮った画像も加えた（立ち上げ時に良品を流して集める運用を想定）。
8. **撮影と判定を ROS 2 のノードに分ける。**判定AIを GPU のある別のPCに置く、カメラやロボットを入れ替える、といった現場での構成変更がしやすい。

4. 評価

10ロット×8個＝80個（うち不良30個、上面以外の面にだけ不良がある部品24個）を、同じ部品・同じ不良で次の方式に流しました。判定は学習済みの AI です。

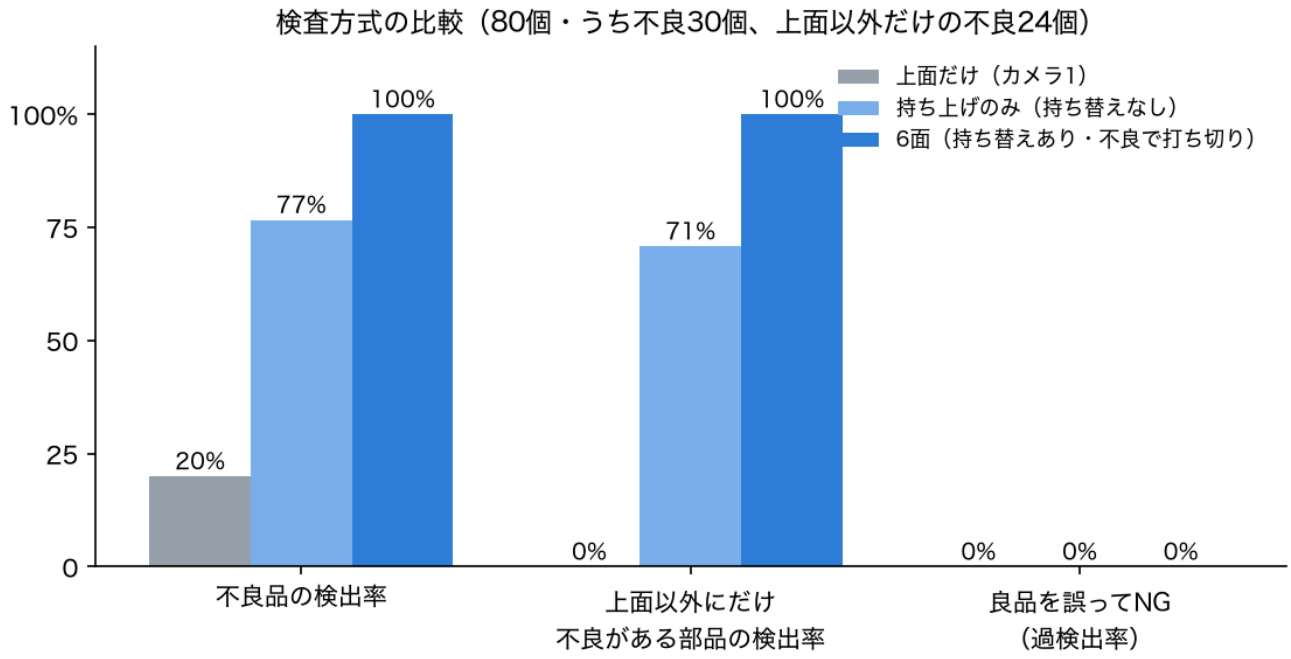


図3　検査方式ごとの不良品検出率と過検出率

方式	不良品の検出率	上面以外だけの不良	過検出率	見逃し	アーム作業時間（平均）	処理能力（1台）
上面だけ（カメラ1）	20%	0%	0%	24個	0.4 秒	5.7 個/分
持ち上げのみ（持ち替えなし）	77%	71%	0%	7個	9.1 秒	4.5 個/分
6面（持ち替えあり）	**100%**	**100%**	0%	0個	15.2 秒	3.2 個/分
6面（打ち切りなし）	100%	100%	0%	0個	15.5 秒	3.2 個/分

- 持ち上げるだけでは、指で隠れた2面の不良（7個）を見逃しました。**持ち替えを入れると見逃しは0個**になりました。
- 6面検査は1台あたり 3.2 個/分です。不良で打ち切ると、打ち切らない場合より作業時間が平均 0.4 秒短くなりました。
- AI 単体の成績は、テスト用の面画像 1332 枚で正解率 100.0%。シミュレーションの不良ははっきり写るため、**見逃しの差は AI の精度でなく「撮っていない面があるか」で決まりました。**

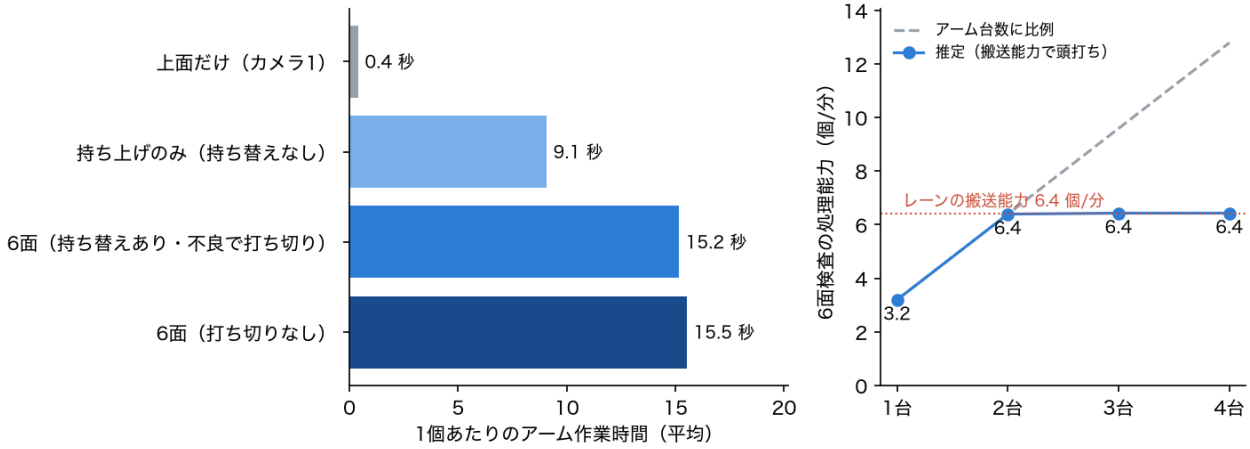


図4　左：方式ごとのアーム作業時間。右：同じセルを並べた場合の処理能力の試算（1台の実測値×台数、ただしレーンの搬送能力が上限）。

アームを並べた場合、2台で 6.4 個/分となり、レーンの搬送能力 6.4 個/分にほぼ達します。それ以上はレーン速度や部品ピッチの見直しが必要です。 公開例でもロボット2台化は1.5倍にとどまっており、共有設備が上限を決める点は同じです。

追加実験：学習していない種類の不良が来たら

分類器の学習から1種類の不良を外し、テストでその不良が来たときに見つけられるかを調べました（面画像単位）。 異常検知は不良画像を一度も使っていません。

学習から外した不良	分類器だけ	異常検知（良品だけで学習）
角欠け	28%	98%
油ジミ	85%	100%
打痕・傷	94%	100%

異常検知の誤報（良品の面を不良とする割合）は 1.2%／面です。ラインに流した結果は次のとおりです（80個、うち角欠けを含む不良 10個）。

判定	不良品の検出率	角欠けの部品の検出率	過検出率
分類器（全種類を学習）	100%	100%	0%
分類器（角欠けを学習していない）	73%	20%	0%
角欠けを学習していない分類器＋異常検知	**93%**	**80%**	16%
全種類を学習した分類器＋異常検知	100%	100%	16%

異常検知を重ねると、未知の角欠けの見逃しは減りました。一方で、ラインで撮る画像は学習用の画像と少しずつ違う（持ち替え後の部品の傾き、照明の当たり方）ため、良品にも反応して誤NGが 16% 出ました。現場で使うなら、異常検知だけが反応した部品は NG 箱ではなく「再検査」に回す運用が合っています。**本ラインの判定は分類器のみ**（誤NG 0%）としました。

5. 結果の読み方と限界

- **試した不良の数：**不良30個で見逃し0個からいえるのは、見逃し率の片側95%上限がおよそ 10% という範囲までです。見逃し率1%以下を示すには、独立した不良品が約300個必要です（二項分布の計算）。
- **把持の補助：**指で挟んだ後は、部品を手先に固定する拘束（溶接拘束）を使って運びます。接触だけではシミュレーション上で部品が指の間を滑ったためです。固定中は指の締め付けを約3 Nに弱め、指が部品にめり込まないようにしました。
- **シミュレーションの画像：**学習も評価も合成画像です。実機では金属面の反射、加工目、照明の当て方、限度ぎりぎりの小さな傷が課題になります（sim-to-real）。先行事例も、面ごとに照明と撮影角度を組にして決めています。
- **検査済みの面への傷：**持ち替え後の指は、すでに検査済みの面に触れます。実機では把持力と爪の材質で、検査後に傷を付けないことを確認する必要があります。
- **処理能力の試算：**複数アームの値は、1台の実測サイクルとレーン能力から計算した推定です。複数台を同時に動かす検証は今後の課題です。

6. ベースにしたサンプルと、発展させた点

課題資料のサンプル「Pick and Place Task（UR3＋2指グリッパ、ROS 2＋Gazebo）」を出発点にしました。 決まった場所の物を決まった場所へ運ぶピック＆プレースを、実際の工場の検査工程として成り立たせるために、次を加えました。

サンプル	本課題
止まった物をつかんで置く	流れてくる部品をつかみ、**動いているレーンの元の位置へ追従して戻す**
つかむことが目的	つかむと隠れる面を、**持ち替えと手首の回転**で見せる。関節の可動限界を見てつかむ向きを選ぶ
AI なし	カメラ3台の画像を AI（ResNet-18）が面ごとに判定。良品だけで学ぶ異常検知も追加実験として比較
動作の確認	検査方式と判定方法を同じ部品で比べ、**見逃し率・過検出率・処理能力**を数値で示す
ROS 2＋Gazebo	ROS 2（撮影ノードと判定AIノードを分離）＋MuJoCo（接触と把持を扱える物理シミュレータ）

7. 使用したもの

項目	内容
主なソフトウェア	MuJoCo、ROS 2 Humble（RoboStack）、PyTorch / torchvision（ResNet-18）、NumPy、imageio、Pillow、matplotlib
ロボットモデル	MuJoCo Menagerie の Franka Emika Panda（Apache-2.0）。リポジトリには含めず、スクリプトで取得。
コード	MIT License。再現手順はリポジトリの README に記載。
生成AIの利用	構想図（図1）の作成と先行事例の調査に生成AIを使用した。コードの作成・デバッグにもAIアシスタントを使った。
動画	デモ部分は2倍速（動画内に表示）。